Worksheet 12 — Software Supply-Chain Security (3 hrs)

> **Course:** Software Security (KOSEN69) · Week 12 **Aligned:** OWASP 2025 — A03 Software Supply Chain Failures · A08 Software or Data Integrity Failures · **CWE:** CWE-1104, CWE-829, CWE-1357, CWE-1395
> **Signature game:** 📦 Dependency Confusion Heist
>
> **Ethics note:** This lab has **no live private/public registry to attack** — the dependency-confusion exercise is a controlled walkthrough (`dependency-confusion.md`) plus the interactive resolver simulation in Task 2, not a real package pull. **Never** plant or publish look-alike packages on the real PyPI or npm — that is an attack on every downstream user, and nothing in this lab requires it.

Part 1 — Student Information

Name	Student ID	Date	Group
Supichaya Rabiabnaweenurak	6631503118	23/10/2569	---

Part 2 — Lecture Questions

1. Explain **dependency confusion** (substitution). Why does a public `acme-internal-utils==99.0.0` win over a private `==1.4.0` when a resolver shops both indexes?
2. How does **typosquatting** (`reqeusts`, `urlib3`) achieve code execution *at install time* before any of your code runs?
3. What is an **SBOM** (CycloneDX/SPDX) and why is it a prerequisite for both incident response and SLSA provenance?
4. Sigstore **keyless** signing uses Fulcio (CA) + Rekor (transparency log) tied to an OIDC identity. Why is that safer than a long-lived private key (CWE-321)?
5. Summarize the **SLSA** levels. Which level does "signed artifact + SBOM + provenance gate before deploy" put you at, and what is still missing?

Answers

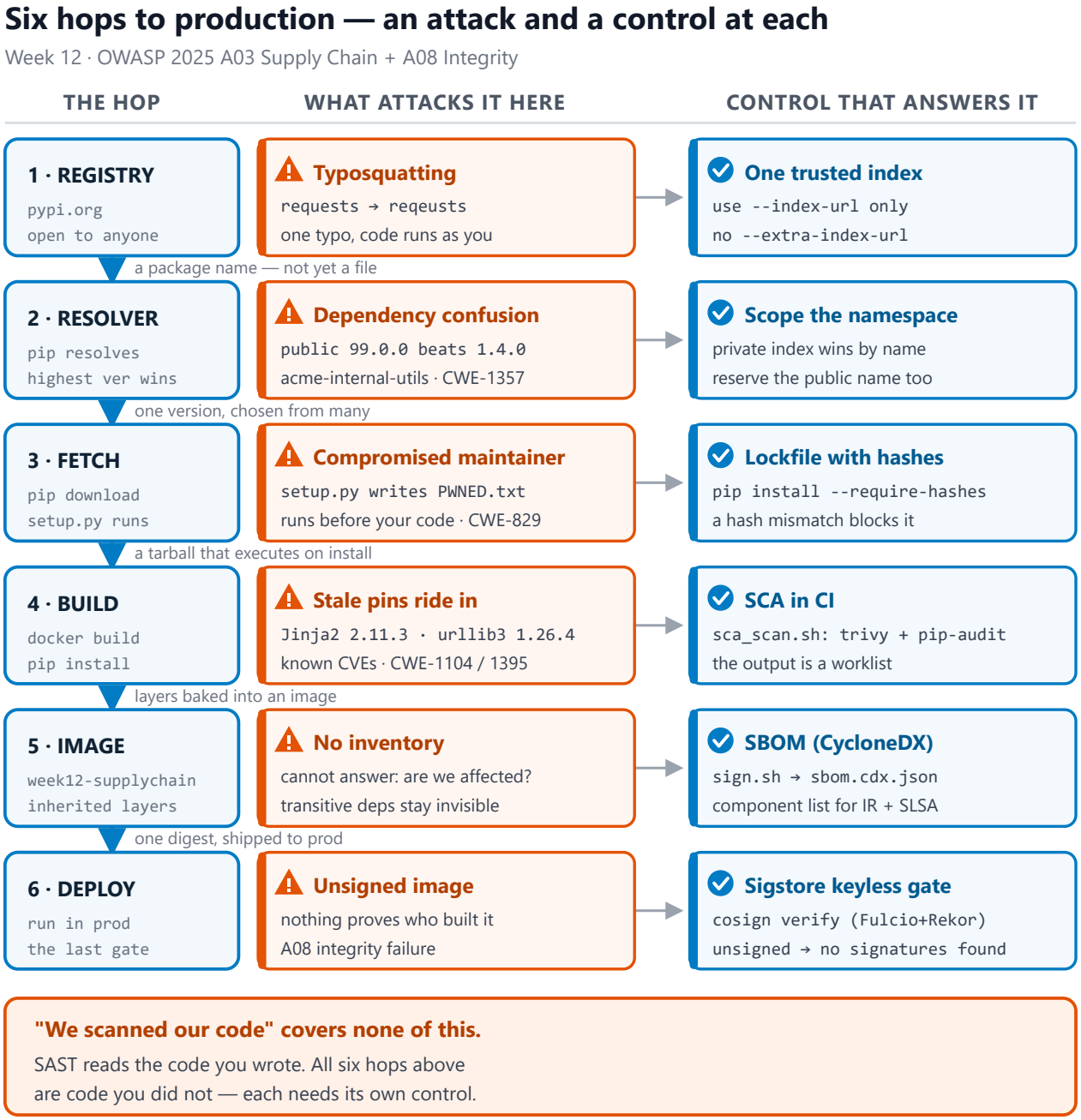
1. **Dependency confusion** is a substitution attack in which an attacker publishes a package with the same name as an internal package but a higher version number. When a resolver is allowed to search both private and public indexes, it compares the candidates and may choose public `acme-internal-utils==99.0.0` over the private `==1.4.0`; the package name alone does not prove which index is trusted.

2. **Typosquatting** uses a look-alike name such as `reqeusts` or `urlib3` to trick a developer or automated dependency update into installing the attacker's package. Package installation can execute build scripts, `setup.py` hooks, custom build backends, or other installer hooks, so malicious code can run before the application's own code starts. The defense is to verify the package name and source, pin versions and hashes, and review or disable install scripts where possible.

3. An **SBOM** is a machine-readable inventory of the components, versions, and sometimes licenses and hashes in a software artifact, commonly using CycloneDX or SPDX. During incident response it identifies whether an affected component is present and which builds must be rebuilt or revoked. It also gives provenance and policy systems a concrete component set to connect to the artifact's build inputs and verify.

4. Keyless Sigstore signing issues a short-lived certificate from Fulcio after authenticating an OIDC identity, and records the signing event in Rekor's append-only transparency log. This avoids storing and distributing a long-lived private key that could be copied and abused indefinitely, reducing the CWE-321 key-management risk and making signing actions auditable. It still depends on protecting the OIDC identity and verifying the certificate identity, issuer, and Rekor evidence correctly.

5. In current SLSA terminology, Build level 1 provides generated provenance, level 2 uses a hosted build service that authenticates the provenance, and level 3 adds a hardened build platform with stronger isolation and provenance that the builder cannot freely falsify. A signed artifact plus an SBOM and a provenance gate is roughly a level-2-style control set if the provenance is authenticated, but an SBOM and signature alone do not establish a SLSA level. Hardened, isolated builds, protected source and builder identities, complete non-falsifiable provenance, and stronger reproducibility or hermetic-build properties are still missing.



Part 3 — Hands-on Lab (150 min)

Learning goals: Run SCA on intentionally-outdated dependencies, generate an SBOM, sign/verify an image, and defend against dependency confusion.

Prerequisites: Docker; the lab folder (`requirements.txt`, `Dockerfile`, `app.py`, `sca_scan.sh`, `sign.sh`). For signing: browser for the OIDC flow + registry push access.

Environment setup

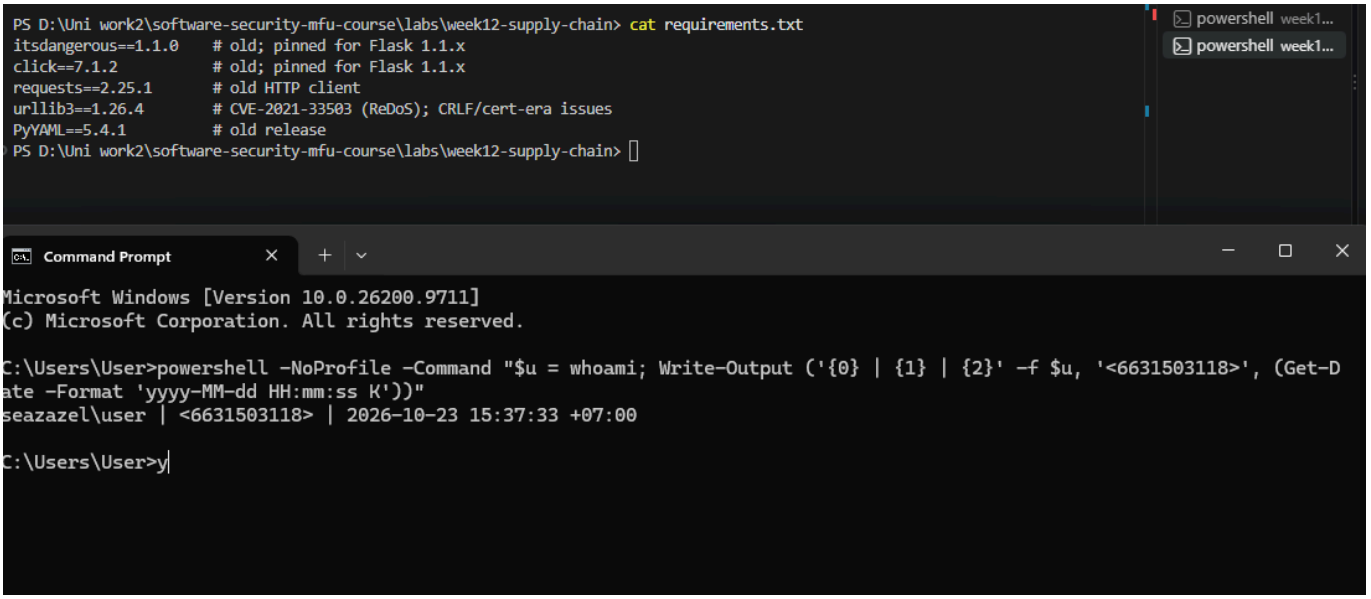
```
cd labs/week12-supply-chain
bash sca_scan.sh                # trivy fs + pip-audit on requirements.txt (+
optional image scan)
docker build -t week12-supplychain:lab .
```

```
bash sign.sh week12-supplychain:lab   # CycloneDX SBOM -> sbom.cdx.json, then
cosign sign + verify
```

What to submit per task: the exact command(s) + output, a screenshot, and a 2–3 sentence remediation note mapping the finding to A03/A08 or the CWE.

Task 0 — Onboarding (15 min)

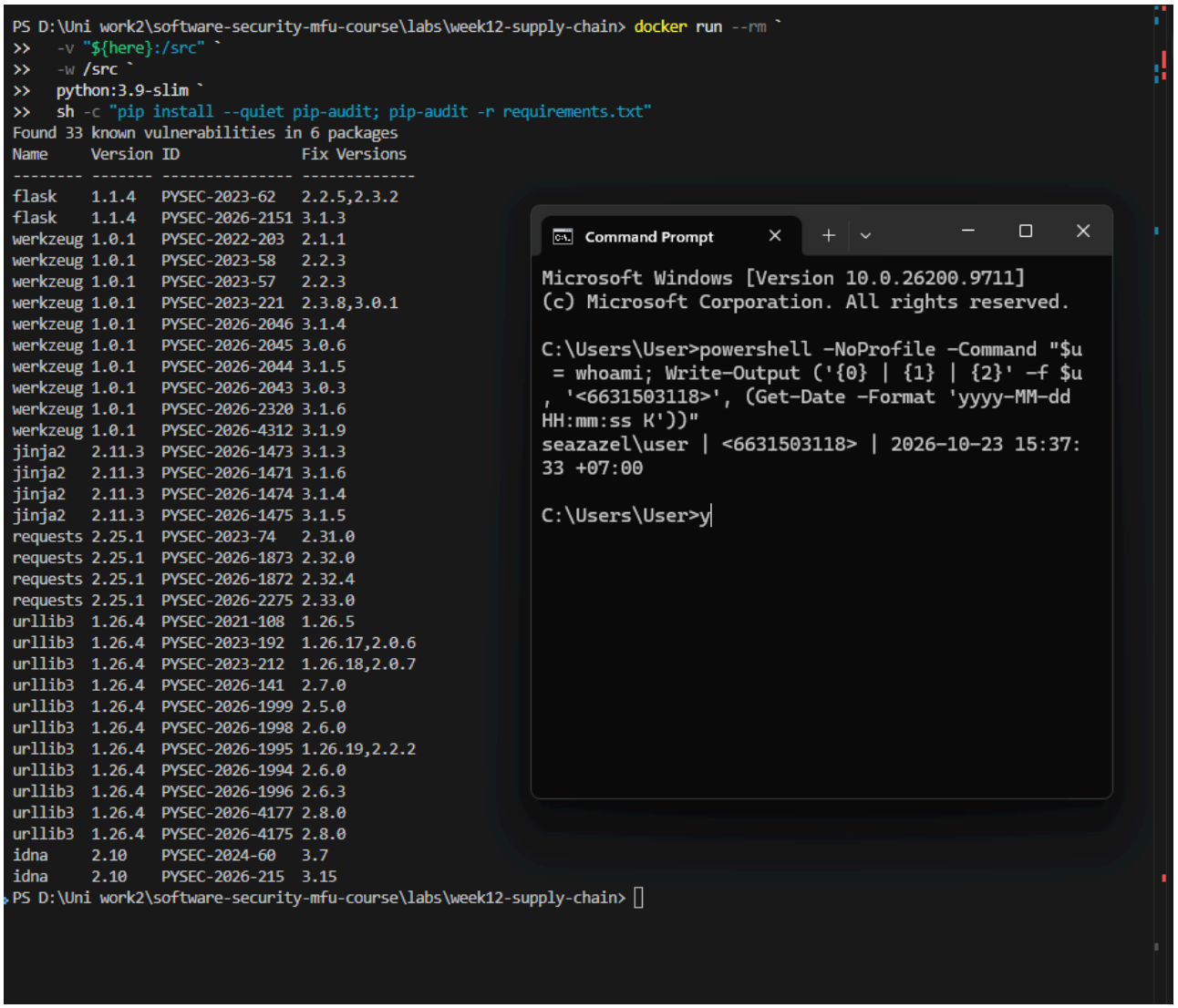
Read `requirements.txt` and list the pinned packages with their versions. Note why they are intentionally outdated (each is deliberately an old release so SCA tools flag its known CVEs). **Deliverable:** the package/version table + which OWASP/CWE this maps to.



Task 1 — SCA scan: build the remediation worklist (35 min)

Goal: Flag the known-vulnerable dependencies — A03. **Steps:**

1. `bash sca_scan.sh` — read the `trivy fs` table (CVE, installed vs. fixed version) and the `pip-audit` advisory IDs (GHSA-/PYSEC-).
2. Pick three findings; record CVE/advisory id, severity, and the fixed version. **Deliverable:** the SCA output + a 3-row remediation table (package → current → fixed).



Package	Current version	CVE / GHSA / PYSEC	Severity	Fixed version	Remediation
1.					Upgrade and retest with SCA.
2.					Upgrade and retest with SCA.
3.					Upgrade and retest with SCA.

Task 2 — Dependency Confusion Heist (35 min)

Goal: Watch the wrong package win — A03 / CWE-1357.

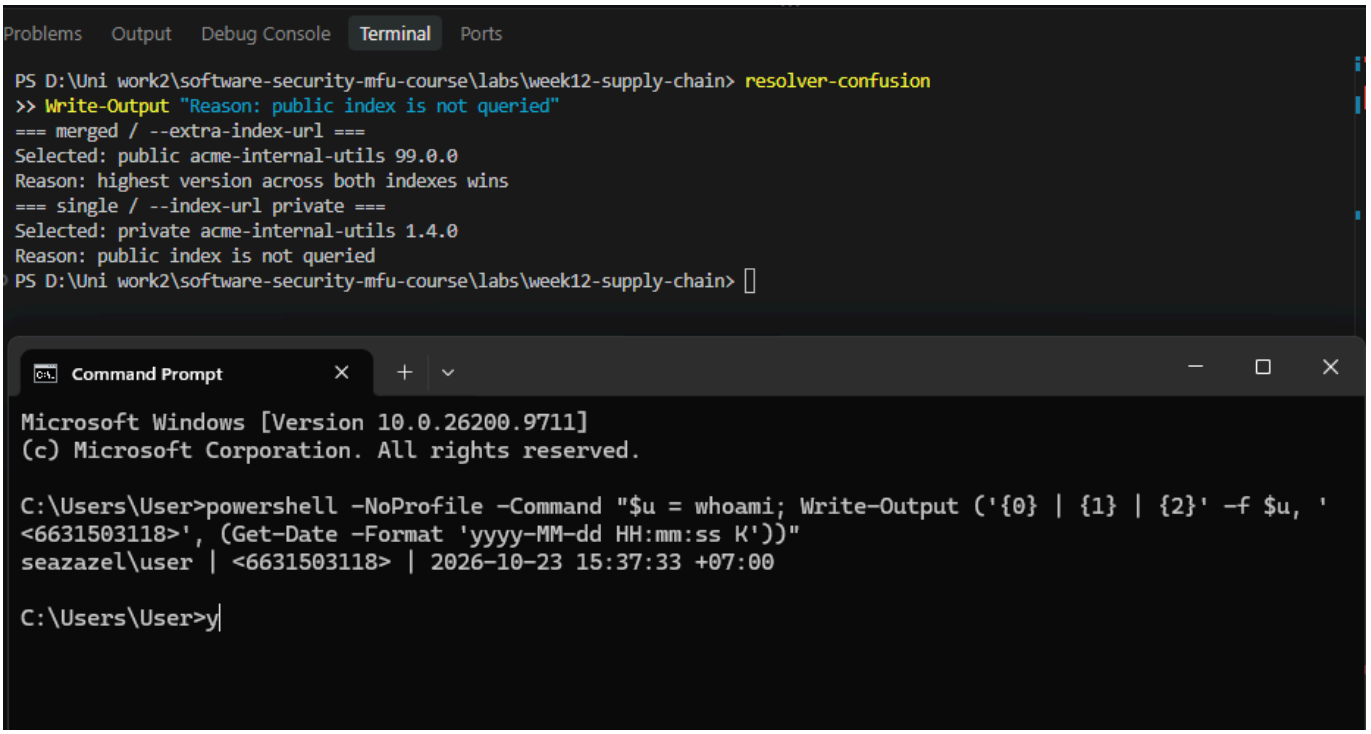
Reality check: this lab does **not** stand up a live private/public registry. `acme-internal-utils` is a worked example, not an installable package — there is nothing to actually `pip install` here (the name doesn't exist on real PyPI either, so don't expect that to work as a shortcut). Task 2 is a controlled walkthrough of the resolver's own rule, using the simulation below, which computes the real "highest version wins" logic live instead of just asserting the outcome.

```
resolver-confusion
```

Steps:

1. Read `dependency-confusion.md`'s "Dependency confusion (substitution)" section for the mechanism.
2. In the simulation, load the **"the attack"** preset (private `1.4.0`, public `99.0.0`, merged / `--extra-index-url` mode) and record which index the resolver picks and why.
3. Keep the same version numbers and switch to the **single-index** (`--index-url`) mode; record how the verdict changes — this is the resolver behavior Task 4's "single trusted index" defense relies on.
4. Try the **"win the race"** preset (private version numerically higher than public) and note why the sim's own explanation calls that "not a defense."

Deliverable: a screenshot of the simulation's verdict in both index modes (merged vs. single) for the same version pair, plus one sentence on why an attacker defeats the "keep my internal version number high" idea.



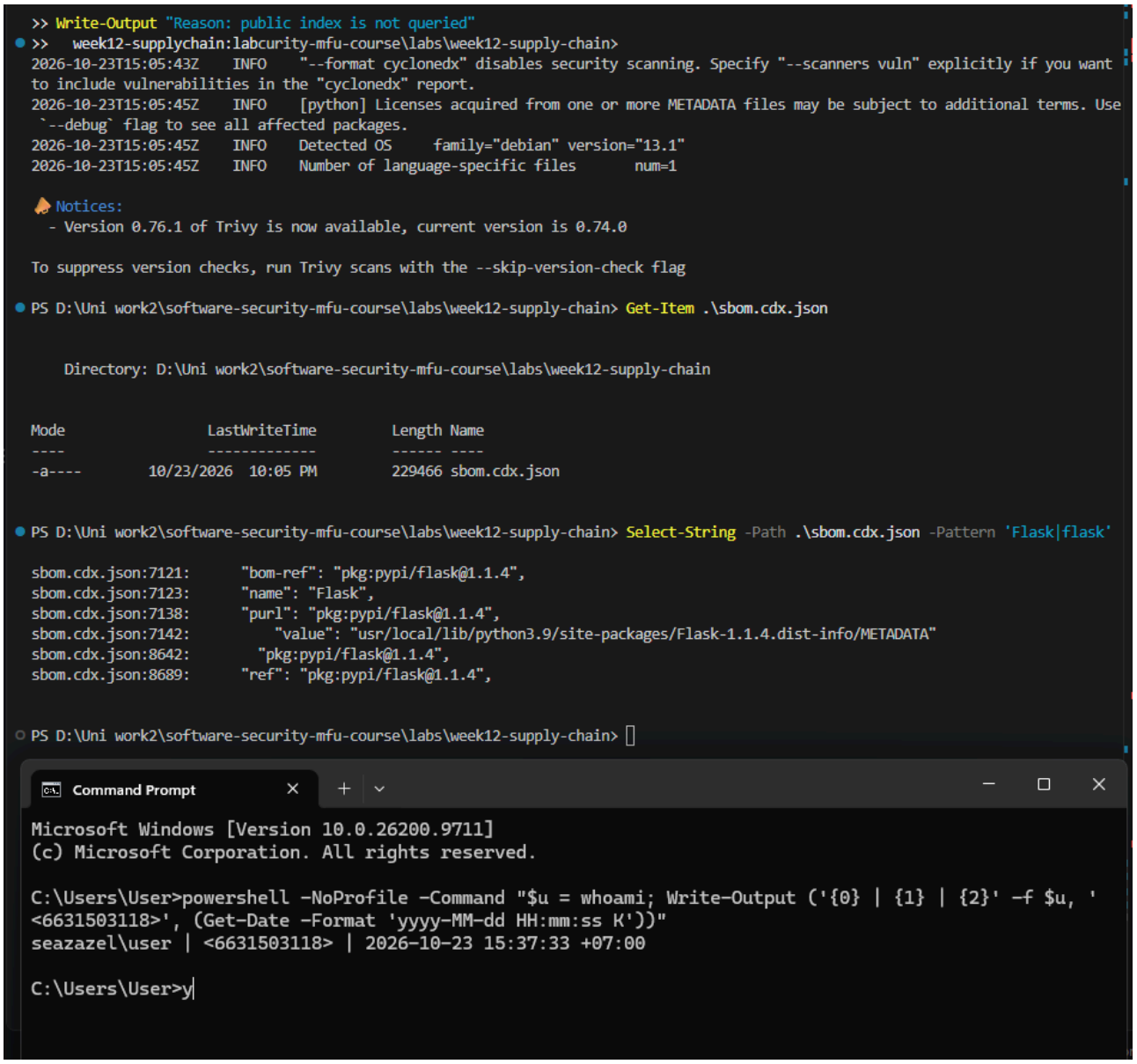
> - The most effective defense is using one trusted private index with --index-url, because the resolver never considers the public look-alike package. Hash-pinned requirements should also be used as defense in depth because a package whose contents do not match the approved hash will be rejected.

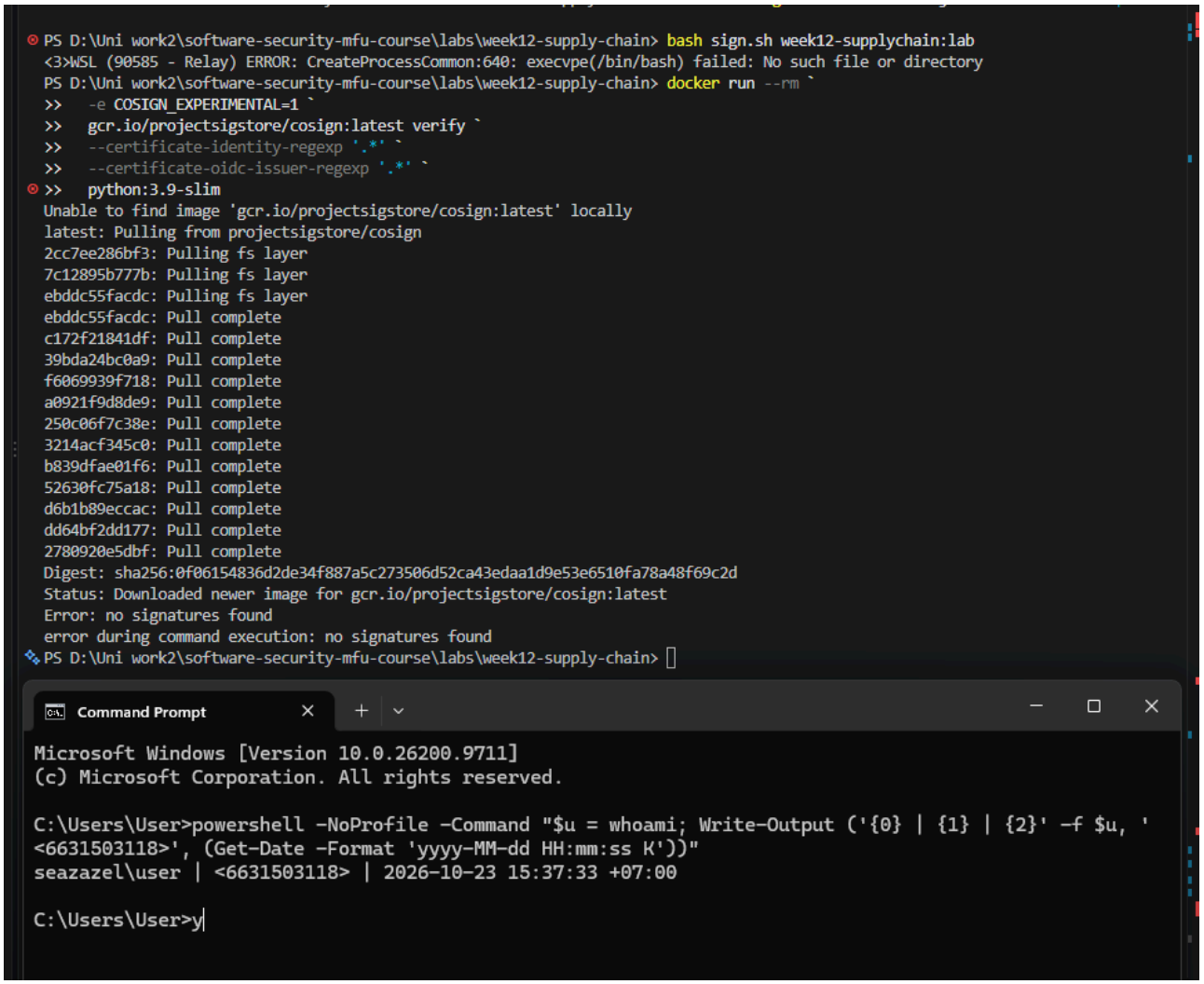
Task 3 — SBOM + signing/verification (30 min)

Goal: Produce a component inventory and prove integrity — A08. **Steps:**

1. After `bash sign.sh week12-supplychain:lab`, open `sbom.cdx.json` and find Flask's entry.
2. Read the `cosign verify` PASS for the signed image.
3. Negative test on an unsigned image (it **must** fail): `cosign verify --certificate-identity-regexp '.*' --certificate-oidc-issuer-regexp '.*' python:3.9-slim` → `Error: no signatures found`. (Both `--certificate-identity*` flags are required in keyless mode; without them cosign stops at `Error: --certificate-identity or --certificate-identity-regexp is required` —

a usage error, which is *not* the same thing as proving the image is unsigned.) **Deliverable:** the SBOM Flask component entry + the verify PASS + the negative-test failure.


>> Write-Output "Reason: public index is not queried"
>> week12-supplychain:labcurity-mfu-course\labs\week12-supply-chain>
2026-10-23T15:05:43Z INFO "--format cyclonedx" disables security scanning. Specify "--scanners vuln" explicitly if you want to include vulnerabilities in the "cyclonedx" report.
2026-10-23T15:05:45Z INFO [python] Licenses acquired from one or more METADATA files may be subject to additional terms. Use `--debug` flag to see all affected packages.
2026-10-23T15:05:45Z INFO Detected OS family="debian" version="13.1"
2026-10-23T15:05:45Z INFO Number of language-specific files num=1
Notices:
- Version 0.76.1 of Trivy is now available, current version is 0.74.0
To suppress version checks, run Trivy scans with the --skip-version-check flag
PS D:\Uni work2\software-security-mfu-course\labs\week12-supply-chain> Get-Item .\sbom.cdx.json
Directory: D:\Uni work2\software-security-mfu-course\labs\week12-supply-chain
Mode LastWriteTime Length Name
-a---- 10/23/2026 10:05 PM 229466 sbom.cdx.json
PS D:\Uni work2\software-security-mfu-course\labs\week12-supply-chain> Select-String -Path .\sbom.cdx.json -Pattern 'Flask|flask'
sbom.cdx.json:7121: "bom-ref": "pkg:pypi/flask@1.1.4",
sbom.cdx.json:7123: "name": "Flask",
sbom.cdx.json:7138: "purl": "pkg:pypi/flask@1.1.4",
sbom.cdx.json:7142: "value": "usr/local/lib/python3.9/site-packages/Flask-1.1.4.dist-info/METADATA"
sbom.cdx.json:8642: "pkg:pypi/flask@1.1.4",
sbom.cdx.json:8689: "ref": "pkg:pypi/flask@1.1.4",
PS D:\Uni work2\software-security-mfu-course\labs\week12-supply-chain>
Command Prompt
Microsoft Windows [Version 10.0.26200.9711]
(c) Microsoft Corporation. All rights reserved.
C:\Users\User>powershell -NoProfile -Command "$u = whoami; Write-Output ('{0} | {1} | {2}' -f $u, '<6631503118>', (Get-Date -Format 'yyyy-MM-dd HH:mm:ss K'))"
seazazel\user | <6631503118> | 2026-10-23 15:37:33 +07:00
C:\Users\User>y

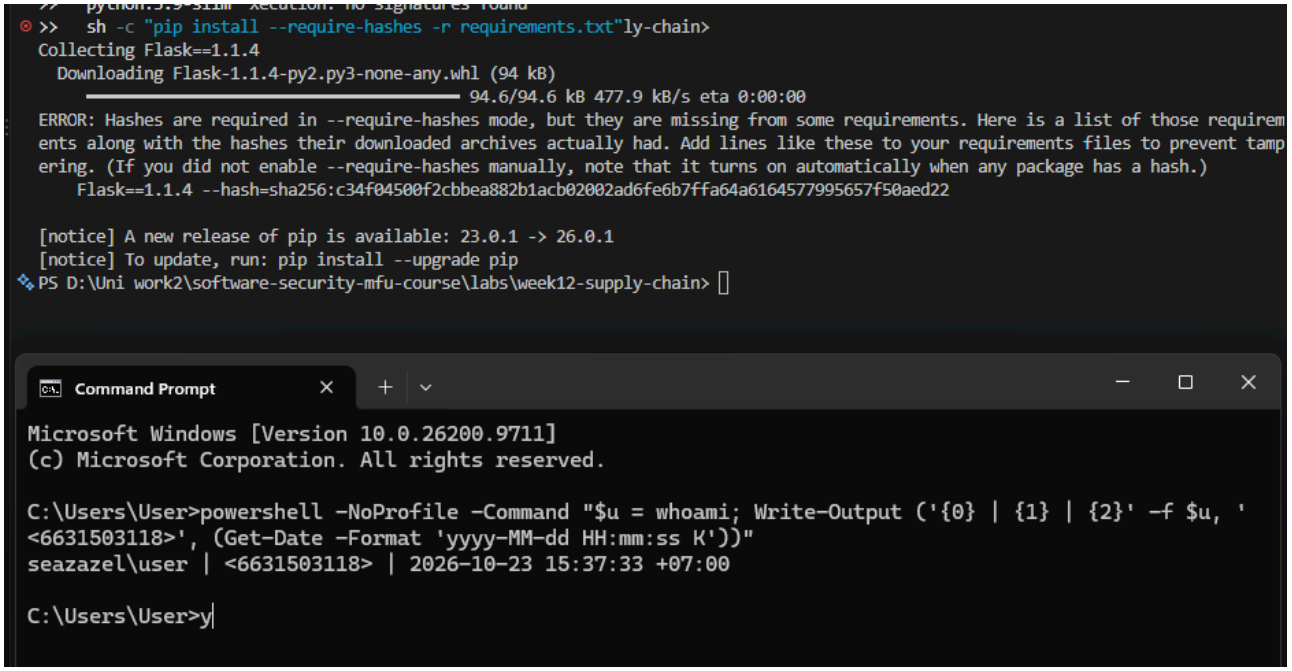


Task 4 — Defend / fix it (35 min)

Goal: Stop dependency confusion and lock integrity — defenses from `dependency-confusion.md` + `sign.sh`. **Steps:**

1. **Pin + hashes:** run `pip install --require-hashes -r requirements.txt` against this lab's own `requirements.txt` (it has no hashes yet) and record pip's refusal — `ERROR: Hashes are required in --require-hashes mode...` plus the `--hash=sha256:…` line pip prints for you. That refusal is the same mechanism that would block a real confusion substitution: once hashes are required, a package that doesn't match — from *either* index — cannot install silently.
2. **Single trusted index:** use one `--index-url` instead of `--extra-index-url`; explain why the resolver stops shopping around (tie this back to Task 2's single-index verdict).
3. **Namespace scoping:** describe reserving/namespacing the internal package name.
4. **Provenance gate:** state how the `cosign verify` from `sign.sh` becomes a gate before a simulated deploy. **Deliverable:** the `--require-hashes` refusal output (step 1) + the simulation's before/after

verdict for merged vs. single index (from Task 2) + the one defense you found most effective and why.



> - --index-url selects one trusted package index. Unlike --extra-index-url, pip does not merge candidates from a public and private index, so a public look-alike package cannot win by having a higher version. No real private registry was provided for this lab, so this defense is documented rather than executed.

Part 4 — Reflection

1. **Mapping:** table — finding | tool (`sca_scan.sh` / `sign.sh`) | OWASP 2025 id | CWE | fix.
2. **Real breach:** analyze the **XZ Utils backdoor (CVE-2024-3094)**. Which OWASP/CWE applies, and would an SBOM + signing + provenance gate have caught a maliciously-modified upstream dependency? (Compare with SolarWinds or Log4Shell if useful.)
3. **Best mitigation + SLSA self-assessment:** one paragraph — which SLSA level your defenses reach and why, and the single highest-leverage control for your team.

Reflection answers

Finding	Tool	OWASP 2025 id	CWE	Fix
Outdated vulnerable dependencies	`sca_scan.sh` / Trivy / pip-audit	A03 Software Supply Chain Failures	CWE-1104, CWE-1395	Upgrade to supported fixed versions and keep an SCA gate in CI.
Dependency confusion	resolver simulation	A03 Software Supply Chain Failures	CWE-1357	Use one trusted index, namespace internal packages, and require hashes.

Finding	Tool	OWASP 2025 id	CWE	Fix
Unverified or modified artifact	`sign.sh` / cosign	A08 Software or Data Integrity Failures	CWE-829	Generate an SBOM, sign the artifact, and require verification before deployment.

The XZ Utils backdoor, CVE-2024-3094, was a malicious supply-chain modification introduced through upstream release artifacts and maintainer access. It maps to A03 and A08 because the trusted dependency and its release process were compromised; it can also be discussed as CWE-829 because software incorporated functionality from an untrusted control sphere. An SBOM would help identify which builds contained the affected XZ version, but an SBOM alone would not detect that the code was malicious. A signing and provenance gate could have caught it if the release was not signed by an authorized identity or if the provenance did not match the approved build process; it would not help if the attacker legitimately controlled the signing identity and build pipeline.

These defenses provide a useful SLSA level-2-style baseline when provenance is authenticated and deployment verifies the artifact, but they do not by themselves prove SLSA level 3 hardened, isolated builds. The highest-leverage control is a trusted, isolated build pipeline that generates non-falsifiable provenance and refuses unsigned or unapproved dependencies; SCA, hashes, SBOMs, and signatures then provide layered evidence and policy checks.

Grading rubric (100)

Criterion	Weight
Lecture questions (Part 2)	20
Exploitation + evidence (Tasks 1–3: SCA findings, confusion proof, SBOM/verify)	40
Defense (Task 4: pinning/hashes, single index, scoping, provenance gate)	25
Reflection (Part 4: mapping, XZ breach, SLSA self-assessment)	15
Total	**100**

Evidence & Integrity (required)

- **Identity proof:** every screenshot/diagram must show a terminal running `printf '%s | %s | ' "$(whoami)" '<YOUR-STUDENT-ID>'; date '+%F %T %Z'` **in the same image as the evidence**. When the evidence is a browser page, a DevTools panel or a rendered response, put that terminal **beside the browser and capture the whole screen** — a cropped window carries nothing that identifies you, and the lab's own output is byte-identical for the whole cohort *by design*, so the stamp is the only thing that makes the shot yours. Generic or borrowed evidence is not accepted.
- **Personalized flag (if this lab issues one):** _____ *Flags are unique per student — submitting another student's flag is a violation. How to submit:* ***learn.zcr.ai/submit*** *(full guide: `SUBMISSION.md` in the repo root).*
- **Explain in your own words** *(graded on your reasoning, not copied text):*

1. What did you do, and **why did the vulnerability work**?
2. **Why does your fix actually stop it** — and what could still break it?

🤖 Audit the AI (required)

AI is a power tool you must **distrust** — you are graded on your *critique*, not the AI's answer.

1. Ask an AI assistant to exploit **or** fix this week's vulnerability. Paste its full answer.
2. **Find what's wrong or risky** in it — insecure code, a subtly incomplete fix, a hallucinated API/function/CVE, a missed edge case, or wrong reasoning. Quote the exact line(s).
3. Produce the **correct, verified** version yourself and explain in 2–3 sentences why the AI's output was insufficient.

> Disclose your AI use in the Part 1 table. This task counts toward your **Defense + Reflection** score.

Audit response

The AI guidance presented this as an example configuration:

```
--index-url https://private-registry.example/simple/
```

That is not a complete command, so running it by itself in PowerShell produced a parser error. It also uses a placeholder domain that cannot install packages. The corrected guidance is to explain the option in prose or place it after a real pip command, for example `python -m pip install --index-url $privateIndex -r requirements.txt`, where `$privateIndex` contains an instructor-provided registry URL. Because this lab does not provide a real private registry, I verified the behavior with the local merged-versus-single-index simulation instead of pretending that the placeholder command was a live test.

The earlier instructions also assumed WSL's `bash` was available. In this environment Ubuntu produced a WSL service error, so the verified workaround was to run Trivy and pip-audit through Docker directly from PowerShell. This mattered because the scan failure was an environment failure before the SCA script ran, not evidence that the dependencies were safe.

🧠 Comprehension & Prompt (required)

A. Explain in Plain English (EiPE). In 2–3 sentences, in your own words, describe what this week's vulnerable code/endpoint actually *does* and *why it is exploitable* — explain the mechanism, don't dump jargon.

B. Prompt Problem. Write a **single prompt** that makes an AI produce a *correct, secure* fix for one finding. Run it: does the exploit now fail? If not, refine the prompt and try again. Submit the **final prompt + the verified result**. *Graded on the prompt's precision and your verification — this trains problem decomposition and AI literacy (Denny et al. 2024).*

Comprehension answers

A. Explain in Plain English: The vulnerable dependency setup allows a resolver to search both a private and a public package source and choose the highest version, even when the public package is an attacker-

controlled look-alike. A malicious package can run installation hooks before the application starts, so the attack can compromise the build machine without requiring a vulnerability in the application itself.

B. Final prompt:

> Design a secure CI policy for this Week 12 Python build. Use one approved package index instead of merging public and private indexes, require pinned dependency versions with SHA-256 hashes, generate a CycloneDX SBOM, and require a Sigstore/cosign signature and authenticated provenance before deployment. Do not invent a registry URL or claim a verification pass that was not run. List the exact checks, the failure conditions, and the evidence each check should produce.

Verified result: The local simulation showed that merged indexes select the public `99.0.0` package, while a single private index selects only the private `1.4.0` package. The unsigned-image test returned `Error: no signatures found`, confirming that the verification gate fails closed for an unsigned artifact. The placeholder registry command was not treated as a successful installation because no real private registry exists in this lab.